

Software Design

Chapter 5 Study Notes — Advanced Software Engineering (CS603)

Student: [Your Name] — MCS Candidate

Course: Advanced Software Engineering (CS603), 3 Credit Hours

Instructor: Asst. Prof. Dr. Ali Fahim Ni'ma

Institution: College of Computer Science & Information Technology, University of Wasit

Document type: Seminar / Study notes — English, trial build

Table of Contents

Update field (right-click > Update Field)

(In Word, select all and press F9 — or right-click — to build the table.)

List of Figures

Figure 1. The two-part design process. Conceptual design answers “what” for the customer; technical design answers “how” for the system builders.

Figure 2. Modularity and software cost. Total cost is the sum of a falling development cost and a rising integration cost, so an optimum number of modules exists.

Figure 3. The coupling spectrum, from worst (content) to best (data). Low coupling is a primary design objective.

Figure 4. Common coupling: independent modules read and write a shared global data area, so any change to the variables affects every module.

Figure 5. High versus low coupling visualised as inter-module dependency lines. Fewer, simpler links mean a cheaper system to maintain.

Figure 6. The cohesion ladder, from weakest (coincidental) to strongest (functional). High cohesion is the counterpart objective to low coupling.

Figure 7. A function-oriented structure chart for “Update File”, showing module hierarchy and labelled data couples on the invocation arrows.

Figure 8. Inheritance in object-oriented design: an abstract Shape base class with Square and Triangle subclasses that override Draw().

Figure 9. The object-oriented analysis and design pipeline, from the problem statement through use cases, activity, interaction and class models to design documents.

Figure 10. A UML sequence diagram for the library “Issue Book” scenario, showing the ordered interaction between actor, controllers and databases.

1. Introduction: What Is Design?

Analysis is concerned with establishing what a system must do; design is concerned with how it will do it. Software design is the creative problem-solving activity that transforms a validated set of requirements into a description of the internal structure of the system, sufficiently detailed to guide implementation. Where the requirements specification is a statement of the problem, the design is the first statement of the solution.

The output of design is the Software Design Document (SDD). A design must satisfy two different audiences at once. It must be understandable to the customer, because the customer validates that the intended behaviour has been captured, and it must be precise enough for the implementers, because they build directly from it. A design that pleases one audience and not the other is incomplete.

A well-formed design is expected to be correct with respect to the requirements, complete, understandable at the level of abstraction appropriate to its reader, and maintainable. These four qualities, rather than the mere presence of diagrams, distinguish design work from coding.

2. The Design Framework

Design proceeds as an iterative refinement loop rather than a single step. The designer starts from the initial requirements, gathers and answers the open questions they raise, analyses the information, conceives a high-level solution, refines and documents it, and then validates the result against the original requirements. Validation may feed new questions back into the loop, so the process repeats until the design is stable.

Design also passes through levels of formality. It begins as an informal outline, becomes an informal design, is refined into a more formal design, and finally becomes the finished, documented design from which code is written. Each transition adds precision without changing the underlying decisions.

3. Conceptual and Technical Design

Design is commonly separated into two complementary parts. Conceptual design addresses the customer's view: where data comes from, what happens to it, what the user sees, what choices are offered, and the timing and form of screens and reports. Technical design addresses the builder's view: the hardware configuration, the software needed, communication interfaces, input and output devices, and the overall software and network architecture.

The two are produced for different stakeholders but must remain consistent. A change in the conceptual view (for example, adding a new report) normally forces a corresponding change in the technical view. Figure 1 shows the relationship.

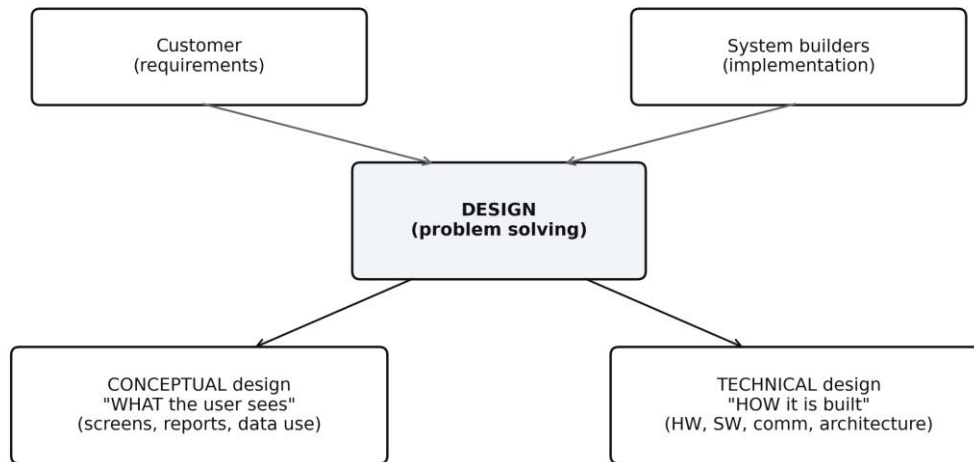


Figure 1. The two-part design process. Conceptual design answers “what” for the customer; technical design answers “how” for the system builders.

4. Modularity

Modularity is the decomposition of a system into a set of discrete, well-defined units called modules. The term has been interpreted at different granularities over time — from a Fortran subroutine, through an Ada package, a C or Pascal function, a C++ or Java class, up to a Java package — but the underlying idea is constant: a module is a unit that can be understood, implemented, tested, and replaced largely in isolation.

A well-formed modular system has modules with a well-defined and single purpose, which can be compiled and stored separately, may use other modules, are easier to use than to rebuild, and are simpler on the outside than on the inside. Modularity pays off because it makes the design intellectually manageable and eases implementation, debugging, testing, documentation, and maintenance.

Modularity is not free, however. Splitting a system into too few modules produces large, unmanageable units; splitting it into too many produces a proliferation of interfaces whose integration cost dominates. The total cost of effort is therefore a U-shaped curve with a minimum at some intermediate granularity, as Figure 2 shows.

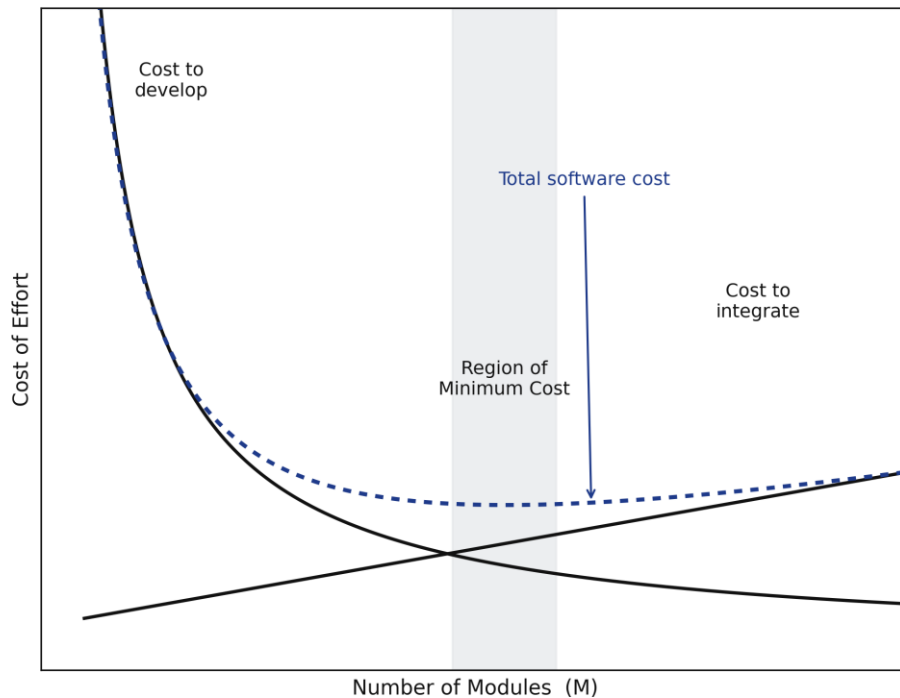


Figure 2. Modularity and software cost. Total cost is the sum of a falling development cost and a rising integration cost, so an optimum number of modules exists.

5. Coupling

Coupling is the degree of interdependence between modules. It measures the strength of the connections that cross module boundaries. Low coupling is desirable because the less one module depends on the internals of another, the more freely either can be changed, tested, or reused without disturbing the rest of the system.

Coupling is not a single condition but a spectrum. From worst to best, the recognised forms are content, common, external, control, stamp, and data coupling. Figure 3 places them on that spectrum.

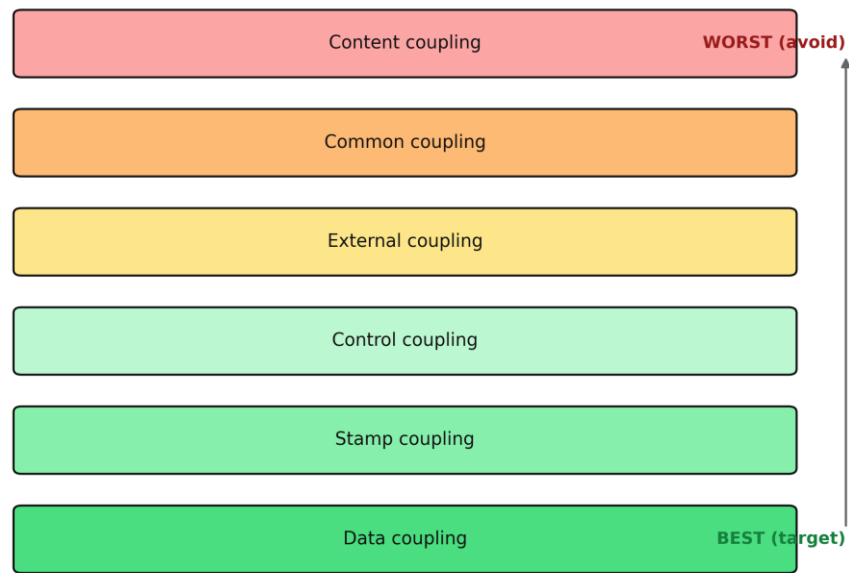
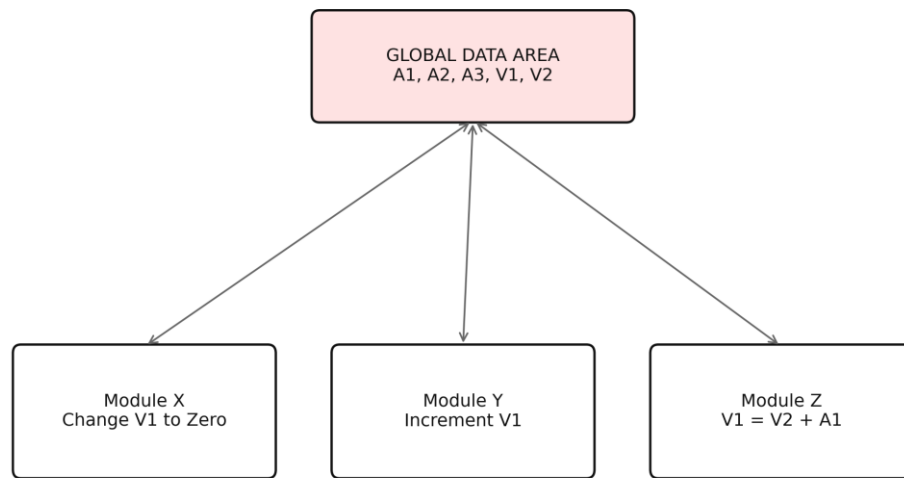


Figure 3. The coupling spectrum, from worst (content) to best (data). Low coupling is a primary design objective.

Content coupling is the worst case: one module reaches into, branches into, or directly modifies the internal data of another. Common coupling arises when independent modules share a global data area, as shown in Figure 4. External coupling ties a module to an externally imposed format or protocol. Control coupling passes a flag whose only purpose is to steer the other module's logic. Stamp coupling passes a whole data structure when only part of it is needed. Data coupling, the best form, passes only the minimal data required, and nothing else.

The practical techniques that reduce coupling are therefore straightforward: pass few parameters, pass data rather than control flags, allow only parent-to-child invocation, and never share global state between peers. Figure 5 contrasts the resulting dependency structure.



All modules read/write the same global variables -> changes ripple everywhere

Figure 4. Common coupling: independent modules read and write a shared global data area, so any change to the variables affects every module.

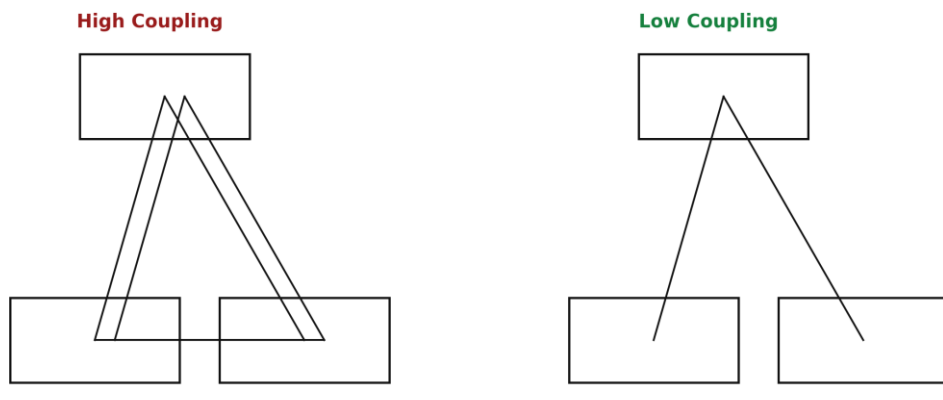


Figure 5. High versus low coupling visualised as inter-module dependency lines. Fewer, simpler links mean a cheaper system to maintain.

Coupling type	What crosses the boundary	Verdict
Content	Access to another module's internals	Worst — avoid
Common	Shared global data area	Poor
External	Externally imposed format/protocol	Poor
Control	A flag that steers logic	Moderate
Stamp	A whole structure, partly unused	Fair

Data	Only the needed data	Best — target
------	----------------------	---------------

6. Cohesion

Cohesion is the strength of the relationships among the elements within a single module. Where coupling looks outward across boundaries, cohesion looks inward. High cohesion is desirable: a module whose parts all serve one purpose is easier to understand, test, and reuse.

Like coupling, cohesion forms a ladder. From weakest to strongest the recognised types are coincidental, logical, temporal, procedural, communicational, sequential, and functional. Figure 6 shows the ladder.

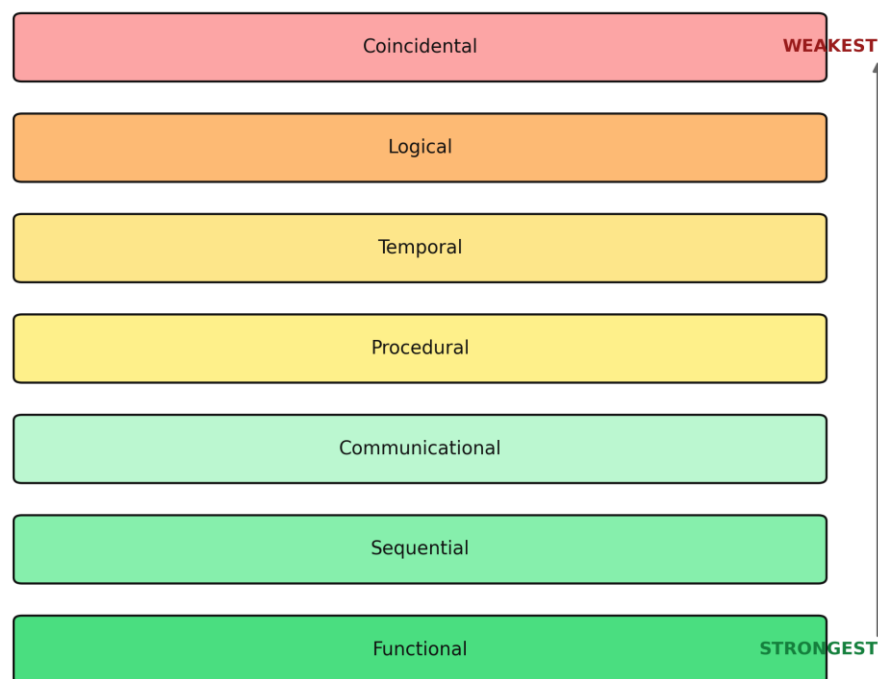


Figure 6. The cohesion ladder, from weakest (coincidental) to strongest (functional). High cohesion is the counterpart objective to low coupling.

Coincidental cohesion packs unrelated statements together with no discernible relationship. Logical cohesion groups elements that are similar in category but controlled by a flag. Temporal cohesion groups activities that happen at the same time, such as initialisation. Procedural cohesion groups steps that must run in a fixed order. Communicational cohesion groups steps that operate on the same data. Sequential cohesion groups steps where the output of one is the input of the next. Functional cohesion, the strongest, groups every element around a single, well-defined task.

The two objectives are complementary and jointly define good modular design: aim for high cohesion within modules and low coupling between them. A design that maximises one without regard to the other is not balanced.

7. Strategies of Design

Once the decision to decompose is made, the designer chooses a direction of refinement. In top-down design, also called stepwise refinement, the designer starts with the most abstract statement of the problem and decomposes it repeatedly until each part is small enough to implement. In bottom-up design, the designer starts from existing low-level modules and composes them upward, which suits situations where a library of reusable parts already exists. In hybrid design, the two are combined.

Top-down lets each module be specialised to exactly one parent and therefore tends to produce very cohesive modules, but it can duplicate logic. Bottom-up favours reuse and generalisation, but modules become shared by several parents and are harder to trace back to a single requirement. The hybrid strategy is the common practical choice: it takes top-down structure from the requirements and bottom-up reuse where components are already available.

8. Function-Oriented Design

In function-oriented design the system is designed from a functional viewpoint: it is seen as a collection of interacting units, each with a clearly defined function. The dominant tool is the structure chart, which partitions the system into black boxes — modules whose functionality is known while their internals are hidden — arranged in a hierarchy with a single root.

A structure chart must not be confused with a flowchart. A flowchart shows the sequence of steps inside an algorithm; a structure chart shows the decomposition of the system into modules and the data and control couples exchanged between them. Alongside the structure chart, function-oriented design uses the data-flow diagram, the data dictionary, and pseudocode. Figure 7 shows a structure chart for a typical “Update File” task; the double-framed box denotes a library (predefined) module, and the labelled arrows denote data couples.

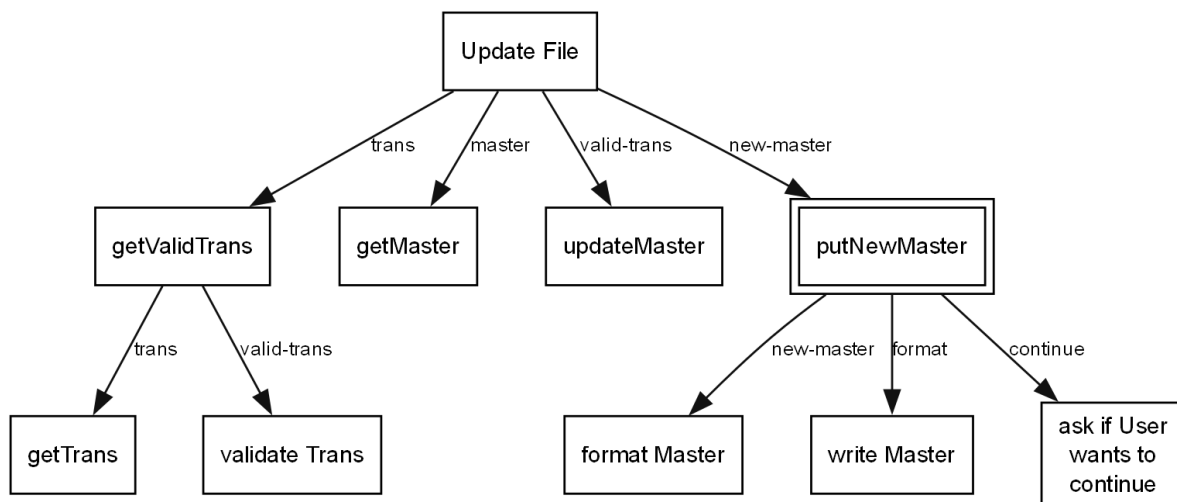


Figure 7. A function-oriented structure chart for “Update File”, showing module hierarchy and labelled data couples on the invocation arrows.

Functional structure may be organised in layers. The top layer coordinates the task, the middle layers perform the main transformations, and the bottom layers provide primitive file and device operations. This layering is what allows the bottom layers to be written once and reused by many tasks.

9. Object-Oriented Design

Object-oriented design is orthogonal to function-oriented design. Instead of organising the system around functions, it organises it around the things in the problem domain — their data (attributes) and their behaviour (operations). The core vocabulary is concise: an object is a run-time instance that holds state and offers operations; a class is the template from which objects are created; a message is a request sent to an object; abstraction suppresses detail; encapsulation hides state behind an interface; inheritance lets a class acquire the attributes and operations of another; polymorphism lets different classes respond to the same request in their own way; and hierarchy organises classes into ranked relationships.

Inheritance and polymorphism are usually discussed together because inheritance without overriding is little more than code sharing. Figure 8 shows a base class Shape with two subclasses, Square and Triangle, each of which supplies its own Draw() operation.

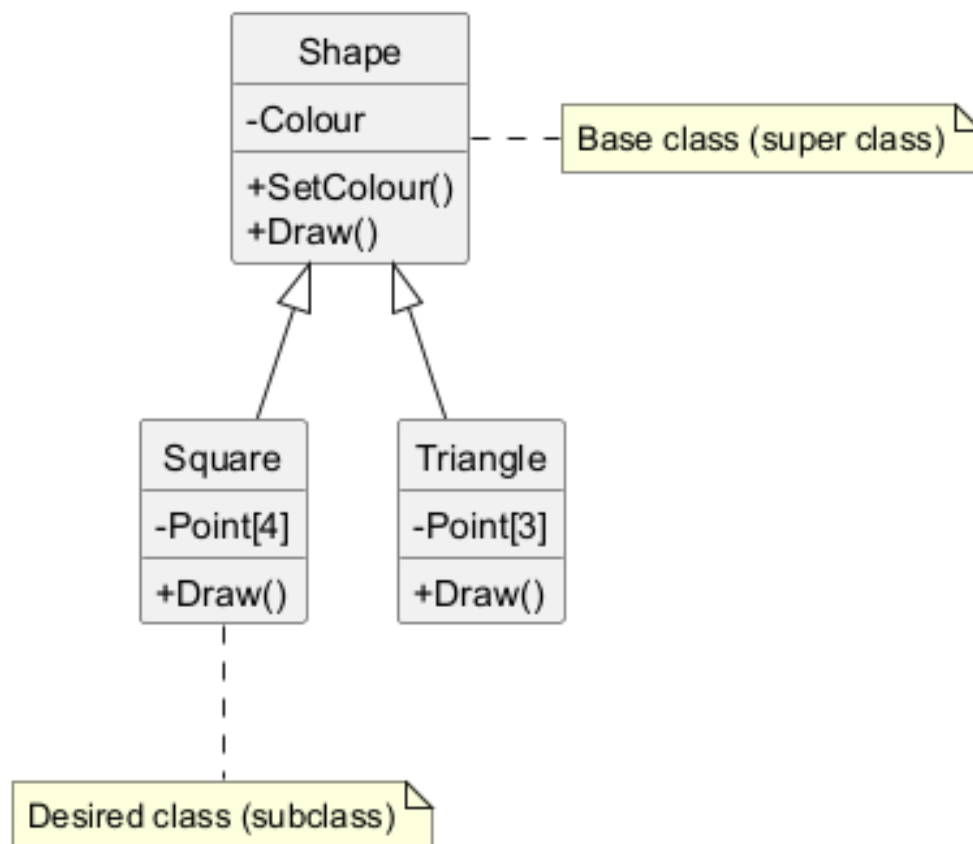


Figure 8. Inheritance in object-oriented design: an abstract Shape base class with Square and Triangle subclasses that override Draw().

The move from requirements to an object-oriented design follows a recognisable pipeline: build a use-case model, draw activity diagrams where control flow matters, draw interaction diagrams (sequence and

collaboration), draw the class diagram, add state or object diagrams where an object has significant state, and finally draw component and deployment diagrams as the design approaches implementation. Figure 9 shows this pipeline.

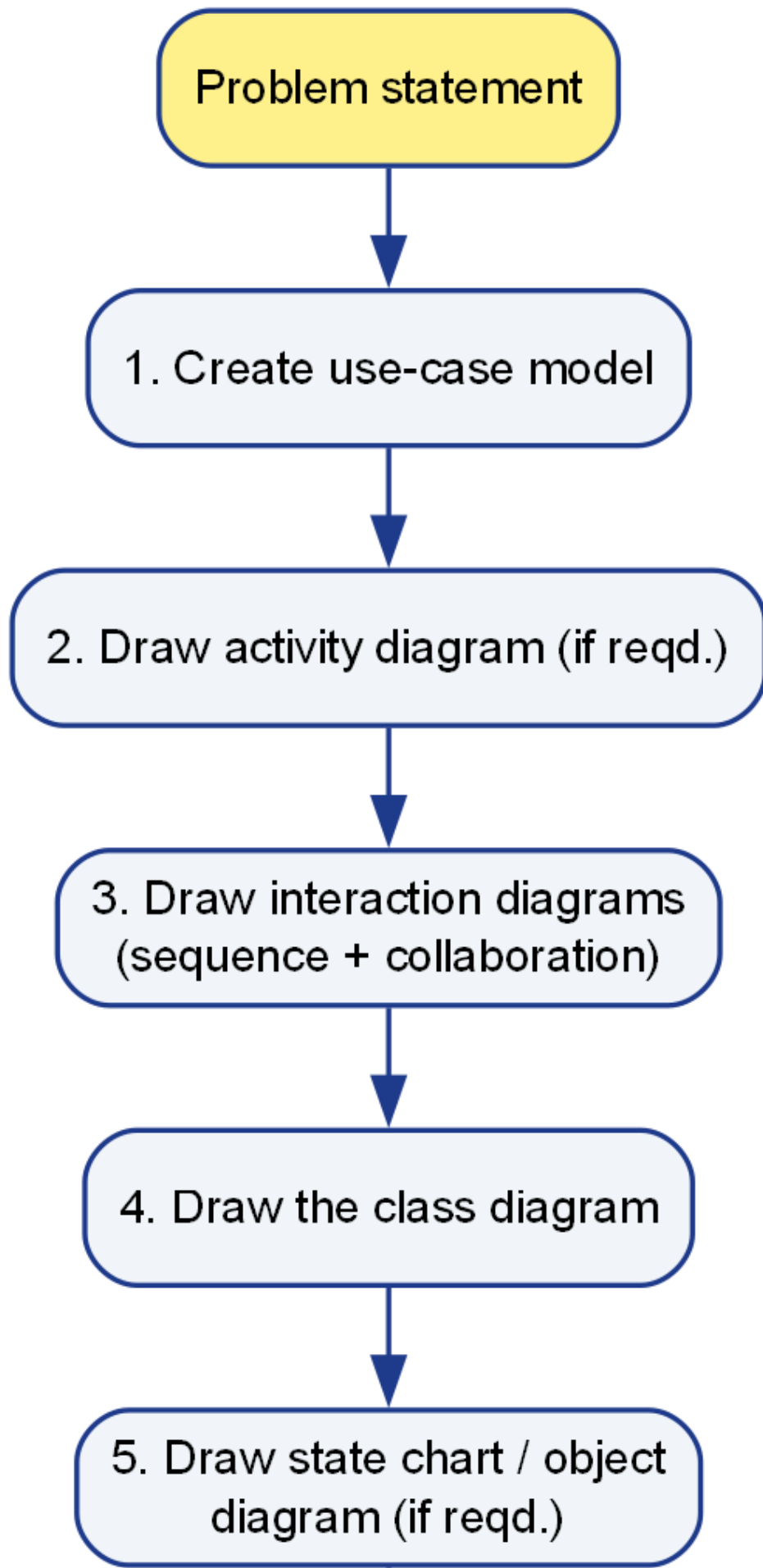


Figure 9. The object-oriented analysis and design pipeline, from the problem statement through use cases, activity, interaction and class models to design documents.

Aspect	Function-oriented	Object-oriented
Unit of decomposition	Function	Object / class
Basic building block	Module	Class
State	Shared, often global	Encapsulated in objects
Extension mechanism	Add a function	Add / override a class
Typical diagram	Structure chart, DFD	Class, sequence, state
Best at	Algorithmic, process-driven systems	State-rich, evolving systems

10. Documenting the Design: The SDD

The design phase concludes with the Software Design Document (SDD), defined by IEEE 1016. The SDD is the medium for communicating the design, the blueprint from which implementation proceeds, and the artefact against which the design is reviewed. It documents design entities, design views, and the attributes of each entity.

Each design entity is described through a consistent set of attributes: identification, type, purpose, function, subordinates, dependencies, interface, resources, processing detail, and data. The document is organised into four standard views: the decomposition view (how the system is partitioned into entities), the dependency view (how entities relate and use resources), the interface view (everything needed to use each entity), and the detail view (the internal detail of each entity).

11. Worked Example: The Library System

The chapter illustrates the design techniques on a university library system. Its three functions are to issue a book, return a book, and answer queries and produce reports such as availability by title or author, printouts, and a logged, password-protected, fully backed-up data store.

Figure 10 traces the “issue book” scenario as a sequence diagram. The operator reads the student's barcode, a reader submits it to a controller, the controller retrieves and validates the student's details against the student database, the operator then reads the book's barcode, and a second controller checks, updates and records the issue. The numbering shows the strict ordering that design must preserve.

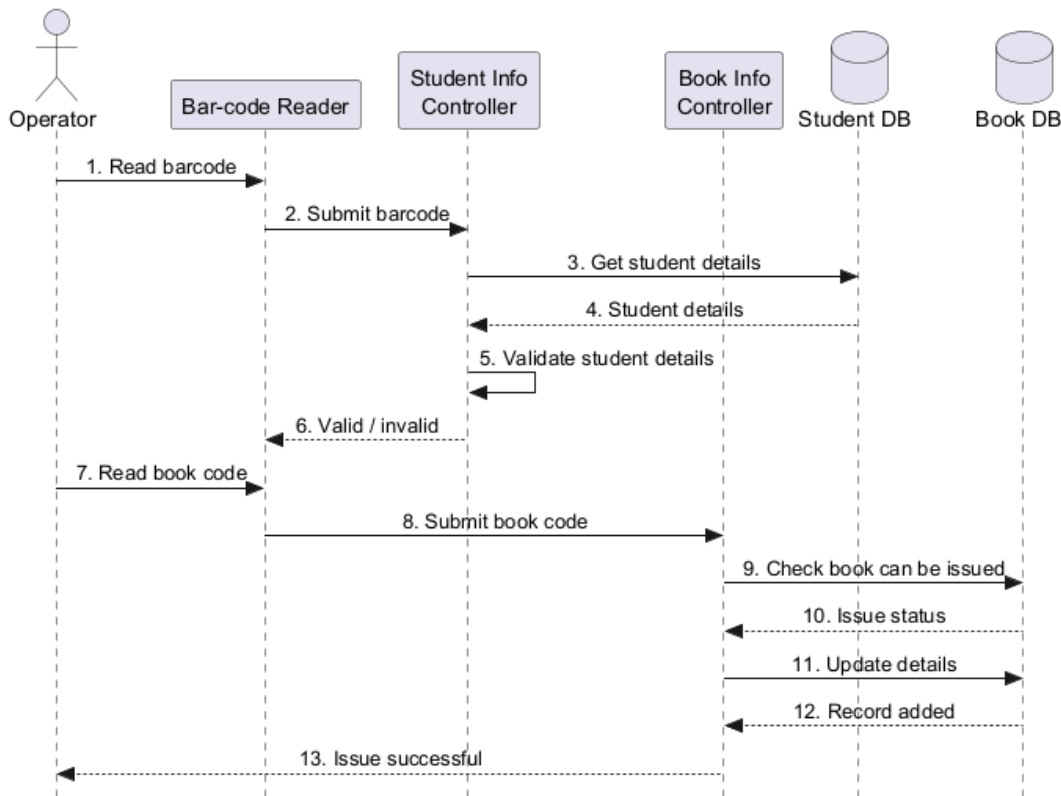


Figure 10. A UML sequence diagram for the library “Issue Book” scenario, showing the ordered interaction between actor, controllers and databases.

The same scenario can be expressed as a set of collaborating objects (controllers for student and book information, entity objects for students and books, and interface objects for the forms), as a class diagram of those objects, and as a state chart for the Book class as it moves between available, issued, and returned states. The choice of view depends on which aspect of the design is being communicated, not on the size of the system.

12. Summary

- Design answers how, where analysis answers what; its output is the SDD.
- Modularity decomposes the system and trades development cost against integration cost.
- Coupling is inter-module interdependence; keep it low (data coupling is best).
- Cohesion is intra-module relatedness; keep it high (functional cohesion is best).
- Design may proceed top-down, bottom-up, or hybrid; hybrid is the practical default.
- Function-oriented design uses structure charts; object-oriented design uses classes and the use-case to deployment pipeline.
- IEEE 1016 governs the SDD, organised into decomposition, dependency, interface, and detail views.

References

- [1] K. K. Aggarwal and Y. Singh, Software Engineering, 3rd ed. New Delhi, India: New Age International Publishers, 2007, ch. 5 (Software Design).
- [2] R. Mall, Fundamentals of Software Engineering, 4th ed. New Delhi, India: PHI Learning, 2018, ch. 5.
- [3] R. S. Pressman, Software Engineering: A Practitioner's Approach. New York, NY, USA: McGraw-Hill Education.
- [4] I. Sommerville, Software Engineering, 9th ed. Boston, MA, USA: Addison-Wesley, 2011.
- [5] IEEE, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions, IEEE Std 1016-2009.